



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

Internet and Mobile Programming

CEF440

TASK 4: System Modelling and Design

Course Instructor

Dr. Valerie

GROUP MEMBERS

Names	Matricule
FORTEBO ACHAH	FE23A062
ANU ROLANCE AKEM	FE23A013
NEIL AMBAYI MPOCHE	FE23A105
OTANG JANICE	FE23A134
ASHU ALBERIC	FE23A015

TASK 4:

SYSTEM MODELLING AND DESIGN FOR THE IMPLEMENTATION OF AN ADAPTIVE E-LEARNING PLATFORM BASED ON QUALITY OF EXPERIENCE FOR INCLUSIVE ONLINE EDUCATION IN LOW-RESOURCE SETTINGS

1. Context Diagram

A context diagram, also referred to as a Level 0 Data Flow Diagram, provides the highest-level view of a system. It represents the entire platform as a single process and illustrates how external entities interact with it through the exchange of data and information. For the Adaptive E-Learning Platform, the context diagram defines the system boundary and identifies all external entities that send data into or receive data from the system.

1.1 System Process

The central process in the context diagram is labelled Process 0: Adaptive E-Learning Platform. This single process encapsulates all internal system functionality and serves as the point of interaction for all external entities.

1.2 External Entities

Student

The Student is the primary external entity and the main beneficiary of the platform. Students interact with the system by accessing learning materials, taking quizzes and assessments, receiving adaptive content tailored to their network conditions, downloading content for offline use, and providing feedback on their learning experience.

Instructor

The Instructor is responsible for populating the platform with educational content. Instructors upload course materials, create assessments, set learning objectives, and monitor the academic performance of their students through the platform.

Administrator

The Administrator oversees the overall operation of the platform. Administrative responsibilities include managing user accounts, configuring system settings, monitoring system performance, and generating analytical reports.

QoE Monitoring System

The QoE Monitoring System is an external technical entity that continuously supplies the platform with real-time network performance data. This includes bandwidth measurements, device capability information, and network latency readings that enable the platform to make informed content adaptation decisions.

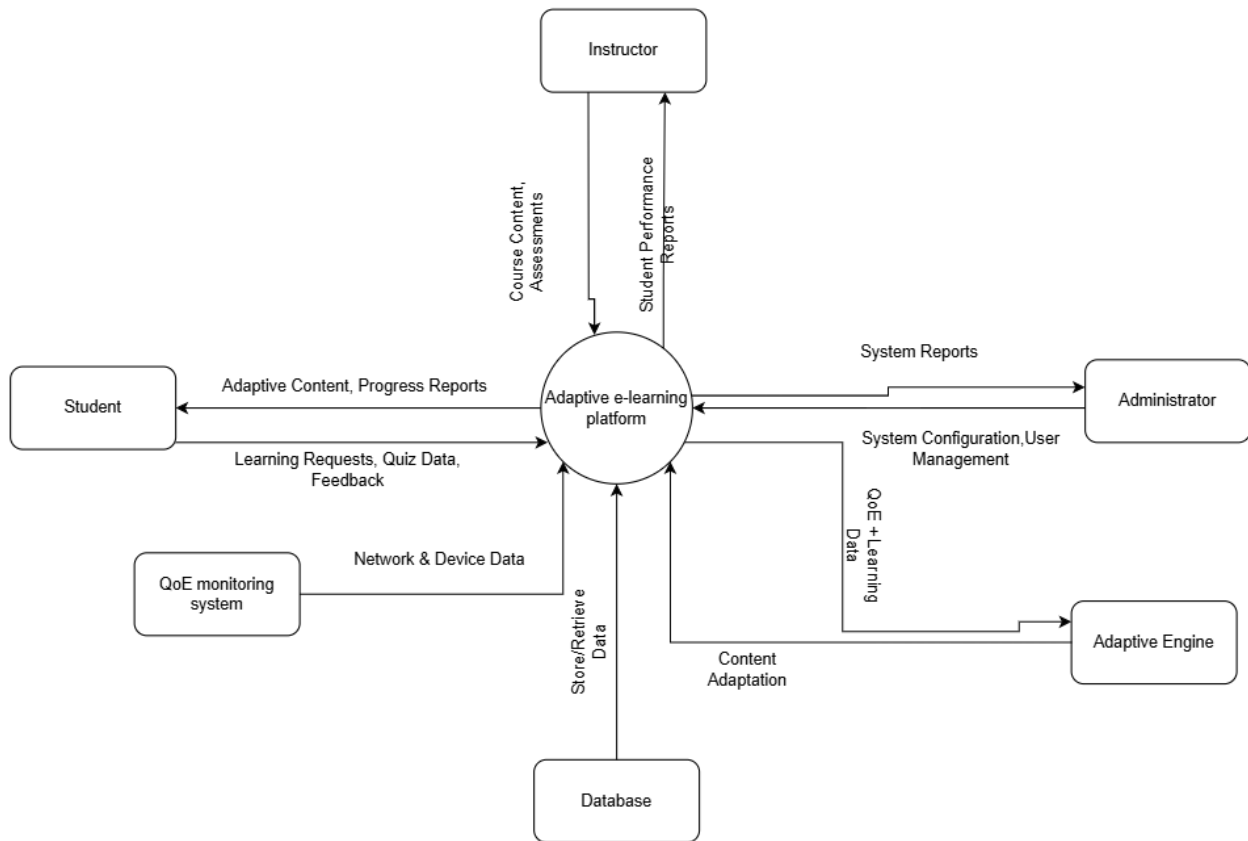
Adaptive Engine

The Adaptive Engine is an external intelligent system that analyses QoE data and learner performance information. It adjusts content quality and format dynamically and generates personalised learning path recommendations for individual students.

Database

The Database serves as the persistent data store for the entire platform. It stores and retrieves user information, course content, assessment records, and system-generated reports as required by the various system processes.

Adaptive E-learning Platform -- Context Diagram



2. Data Flow Diagram (Level 1)

The Level 1 Data Flow Diagram expands upon the context diagram by decomposing the central system process into its core functional modules. It illustrates how data flows between these internal processes, the external entities, and the supporting databases.

2.1 Introduction

The Adaptive E-Learning Platform is a specialised educational system engineered to bridge the digital divide for students in regions with constrained internet connectivity. Unlike conventional Learning Management Systems that depend on high-speed persistent connections, this platform prioritises asynchronous access, data compression, and local caching strategies to ensure academic continuity regardless of network stability.

2.2 Stakeholder Analysis and Interactions

The system serves three primary actors, each with distinct data flow requirements as defined in the Level 1 DFD. Students are the primary consumers who authenticate securely, download resources for offline study, and participate in academic forums. Lecturers serve as content creators who upload materials and academic recordings while providing support through discussion modules. Administrators are system overseers who manage communication flows and maintain the integrity of the core databases.

2.3 Functional Modules

2.3.1 User Authentication (Process 1.0)

This module implements a security layer that verifies user credentials against the User Database. It employs token-based authentication to minimise data overhead and ensure secure access for all user types.

2.3.2 Course Management (Process 2.0)

This module manages all academic documents and course content. Files are compressed within the Course Materials Database to facilitate faster downloads during periods of peak connectivity or reduced bandwidth.

2.3.3 Video Streaming and Download (Process 3.0)

This module employs adaptive bitrate streaming alongside offline download capabilities. Learning content is stored in the Lecture Recordings Database and can be accessed offline, ensuring uninterrupted study even without an active internet connection.

2.3.4 Notification System (Process 4.0)

This module acts as the central communication hub of the platform. It ensures that announcements stored in the Notifications Database are delivered to users reliably, including after periods of being offline.

2.3.5 Discussion Forum (Process 5.0)

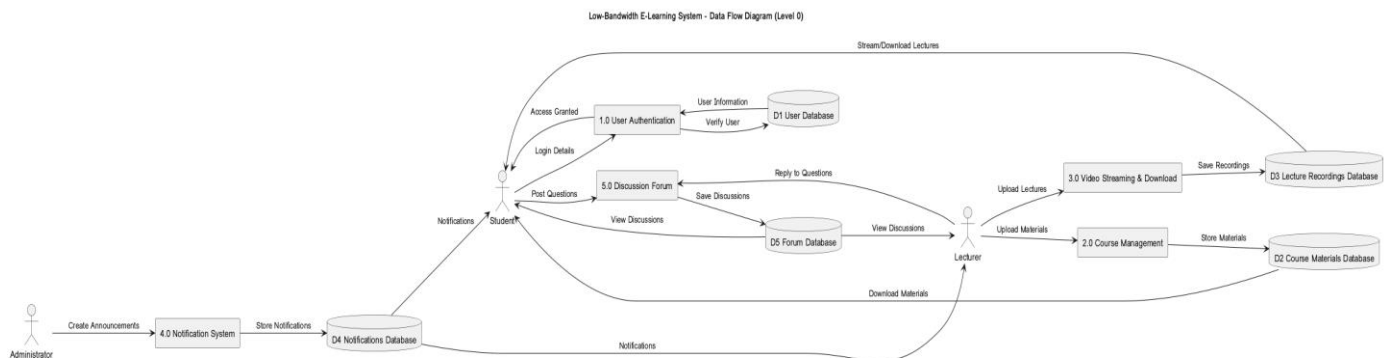
This module facilitates low-bandwidth text-based academic interaction. It creates and maintains a persistent knowledge base of threaded discussions stored in the Forum Database.

2.4 Technical Architecture and Data Persistence

The system utilises five dedicated databases to ensure data integrity and availability. The User Database stores encrypted credentials and user profiles. The Course Materials Database serves as a document repository inclusive of learning paths. The Lecture Recordings Database provides media asset storage with potential CDN integration. The Notifications Database manages alert and announcement queues. The Forum Database maintains a complete history of threaded academic discussions.

2.5 Conclusion

By strategically decoupling content delivery from real-time connectivity requirements, the platform provides a resilient framework for education. Its modular architecture allows for independent scaling of individual services while effectively addressing the core challenges of learning in bandwidth-constrained environments.



3. Use Case Diagram

3.1 Introduction

A use case diagram is a behavioural UML diagram that represents the functional requirements of a system by illustrating the interactions between actors and use cases. For the Adaptive E-Learning Platform, the use case diagram captures all the key functionalities required to deliver adaptive and inclusive online education, particularly for students operating in low-resource environments with limited internet access and basic computing devices.

3.2 Actors

The use case diagram comprises five actors, categorised into human actors and system actors.

3.2.1 Human Actors

The Student is the primary actor of the system, positioned on the left side of the system boundary. Students represent learners who access the platform to enroll in courses, access learning content, and complete assessments. Given that many students in low-resource settings operate under bandwidth and device constraints, the adaptive features of the platform are essential to their learning experience.

The Instructor is positioned on the left side of the system boundary and represents educators who create and manage courses, upload content in various formats, and monitor student progress throughout the platform.

The Administrator is also positioned on the left side of the system boundary and is responsible for managing user accounts, monitoring system performance, generating analytical reports, and configuring adaptive settings and QoE thresholds.

3.2.2 System Actors

The QoE Monitoring System is a system actor positioned on the right side of the system boundary. It represents the external monitoring component that continuously tracks network bandwidth, device capability, and network latency of student devices, supplying this data to enable adaptive content delivery.

The Adaptive Engine is a system actor positioned on the right side of the system boundary. It represents the intelligent processing component that analyses QoE data, selects appropriate content formats, adjusts content quality, and recommends personalised learning paths for students based on their current network and device conditions.

3.3 Use Cases

3.3.1 Student Use Cases

The Student interacts with nine use cases. These include Register or Login, View Course Catalog, Enroll in Course, Access Learning Content, Take Quiz or Assessment, View Progress Report, Receive Adaptive Content, Download Content for offline access, and Rate Content Quality. The

offline download functionality is of particular significance in low-resource settings where persistent connectivity cannot be guaranteed.

3.3.2 Instructor Use Cases

The Instructor interacts with six use cases: Register or Login, Create Course, Upload Content, Set Learning Objectives, View Student Progress, and Manage Assessments. These use cases collectively enable instructors to build, manage, and evaluate educational content on the platform.

3.3.3 Administrator Use Cases

The Administrator interacts with five use cases: Manage Users, Monitor System Performance, Generate Reports, Configure Adaptive Settings, and Manage QoE Thresholds. These use cases provide administrative oversight and control over the operational behaviour of the platform.

3.3.4 QoE Monitoring System Use Cases

The QoE Monitoring System interacts with four use cases: Monitor Bandwidth, Detect Device Capability, Measure Network Latency, and Send QoE Data to the Adaptive Engine. These use cases collectively ensure that accurate real-time network performance data is continuously available to guide content adaptation decisions.

3.3.5 Adaptive Engine Use Cases

The Adaptive Engine interacts with four use cases: Analyse QoE Data, Select Appropriate Content Format, Adjust Content Quality, and Recommend Learning Path. These use cases represent the intelligent decision-making processes that enable the platform to dynamically optimise content delivery for each individual student.

3.4 Relationships

3.4.1 Association

Association relationships connect each actor to their respective use cases through solid lines. Each association indicates that the actor directly participates in and initiates the corresponding use case. Every actor in the diagram maintains at least one association with a use case within the system boundary.

3.4.2 Include Relationships

The include relationship is applied when one use case unconditionally requires another use case to execute. In this system, accessing learning content always triggers QoE monitoring, making the monitoring of bandwidth and latency a mandatory included behaviour. Similarly, the receipt of QoE data by the Adaptive Engine always necessitates the analysis of that data and the selection of an appropriate content format.

3.4.3 Extend Relationships

The extend relationship is applied when one use case optionally augments the behaviour of another under specific conditions. Downloading content for offline use is an optional extension of accessing learning content, triggered only when the student elects to download. Rating content quality is similarly optional. Recommending a learning path extends the adaptive content delivery use case only when the system identifies a need for personalised academic guidance.

3.5 Conclusion

The use case diagram provides a comprehensive overview of the functional requirements of the Adaptive E-Learning Platform. With five actors and twenty-eight use cases, the diagram demonstrates a well-structured system designed to serve the needs of all users. The integration of the QoE Monitoring System and the Adaptive Engine as dedicated system actors underscores the platform's commitment to inclusive, adaptive education that remains accessible regardless of the technical limitations faced by students in low-resource environments.

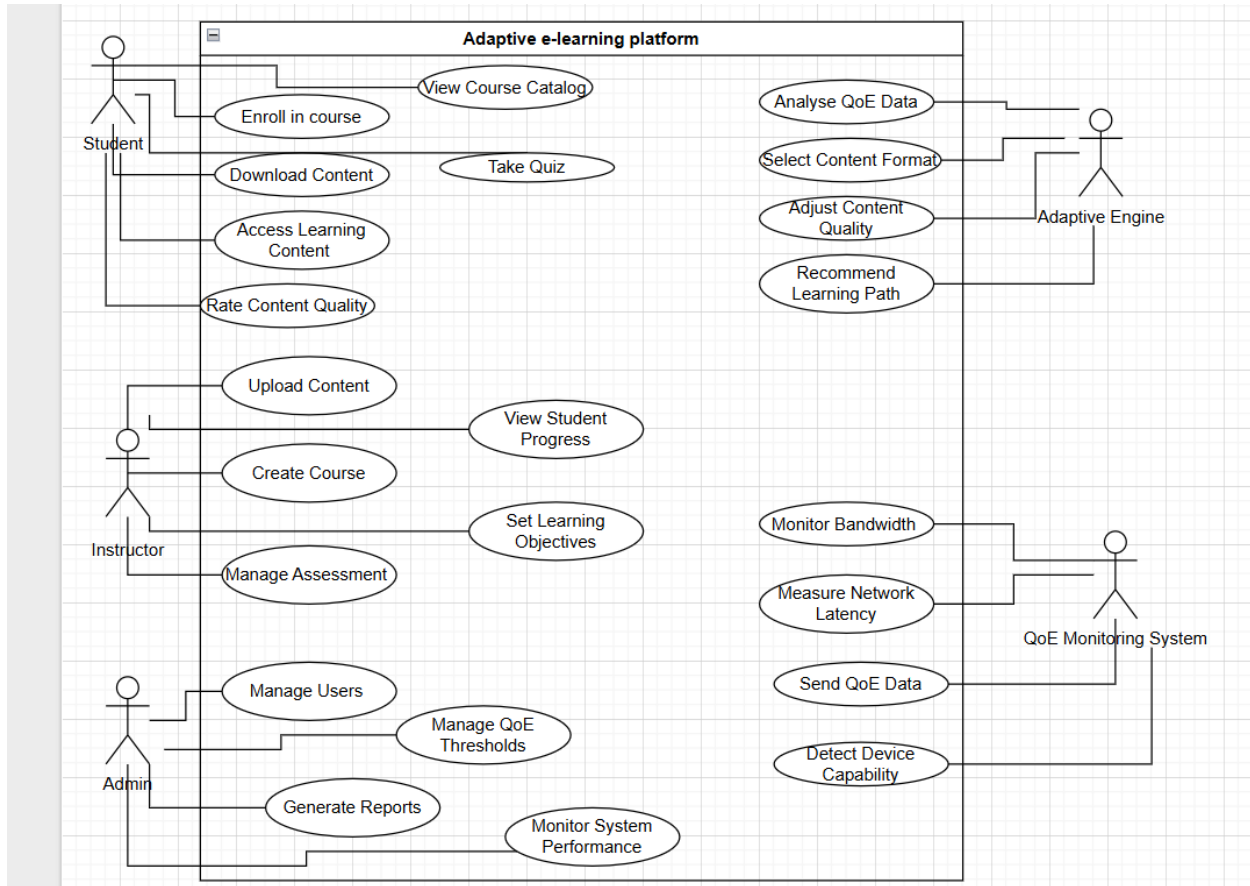


Figure 1: System use case diagram

4. Sequence Diagrams

Sequence diagrams are behavioural UML diagrams that illustrate the interactions between system components over time. The following sequence diagrams were developed for the Adaptive E-Learning Platform to demonstrate how the system responds to various user actions and network conditions.

4.1 Student Login Sequence

The Student Login Sequence Diagram illustrates the authentication process through which students gain access to the platform. The student submits login credentials through the user interface, which forwards them to the Authentication Module. The module validates the credentials against the database and, upon successful verification, grants access and displays the student dashboard. If the credentials are invalid, an appropriate error message is returned to the student.

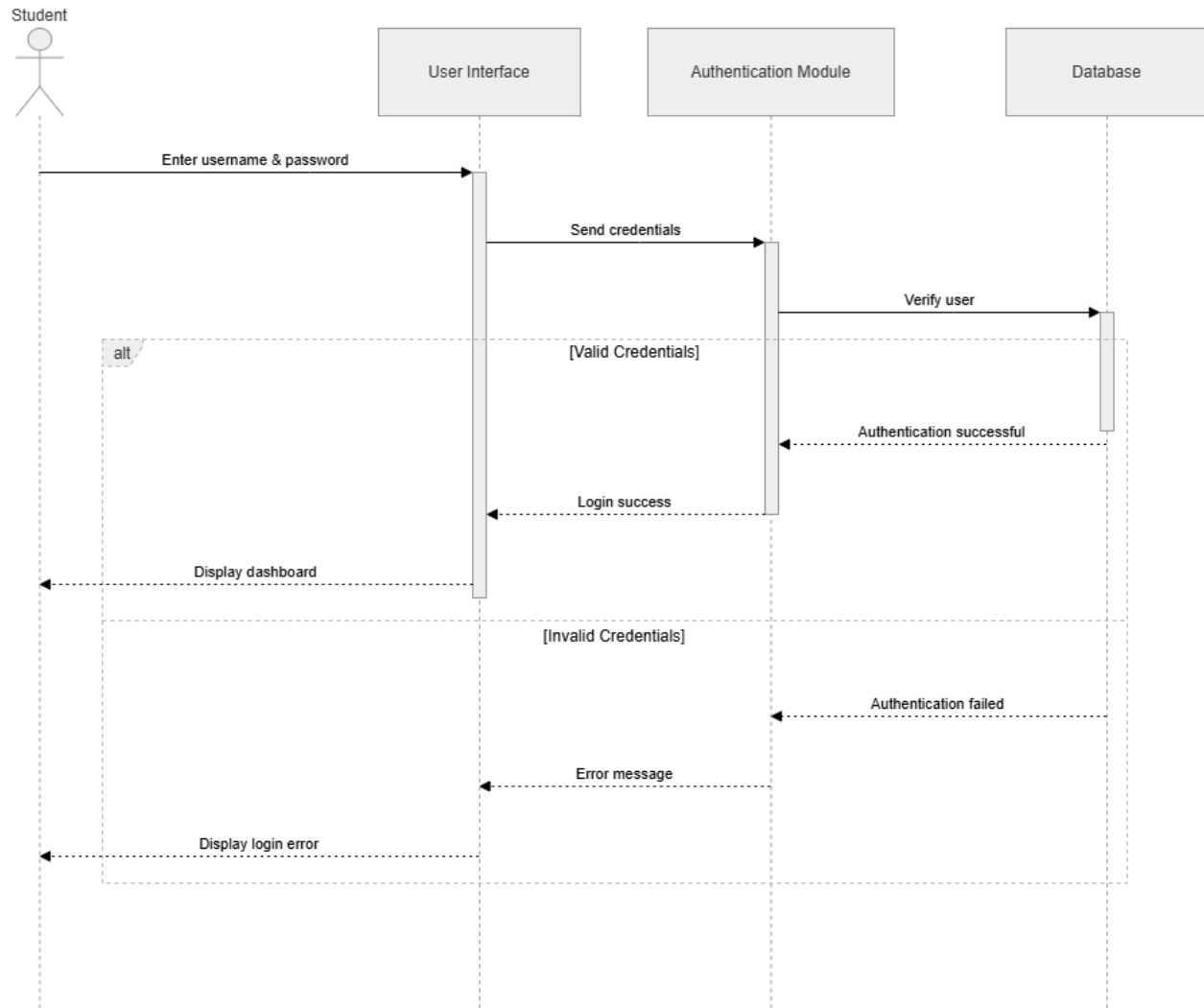


Figure 2: Student login sequence diagram

4.2 Adaptive Content Delivery Sequence

This diagram demonstrates how the system dynamically adapts learning materials in response to varying network conditions. Upon a student's request for content, the Content Delivery Module communicates with the QoE Monitoring System to assess network quality and device capability. The resulting metrics are forwarded to the Adaptive Engine, which retrieves learner history from the database and selects the most suitable content format. Optimised content is then delivered to the student accordingly.

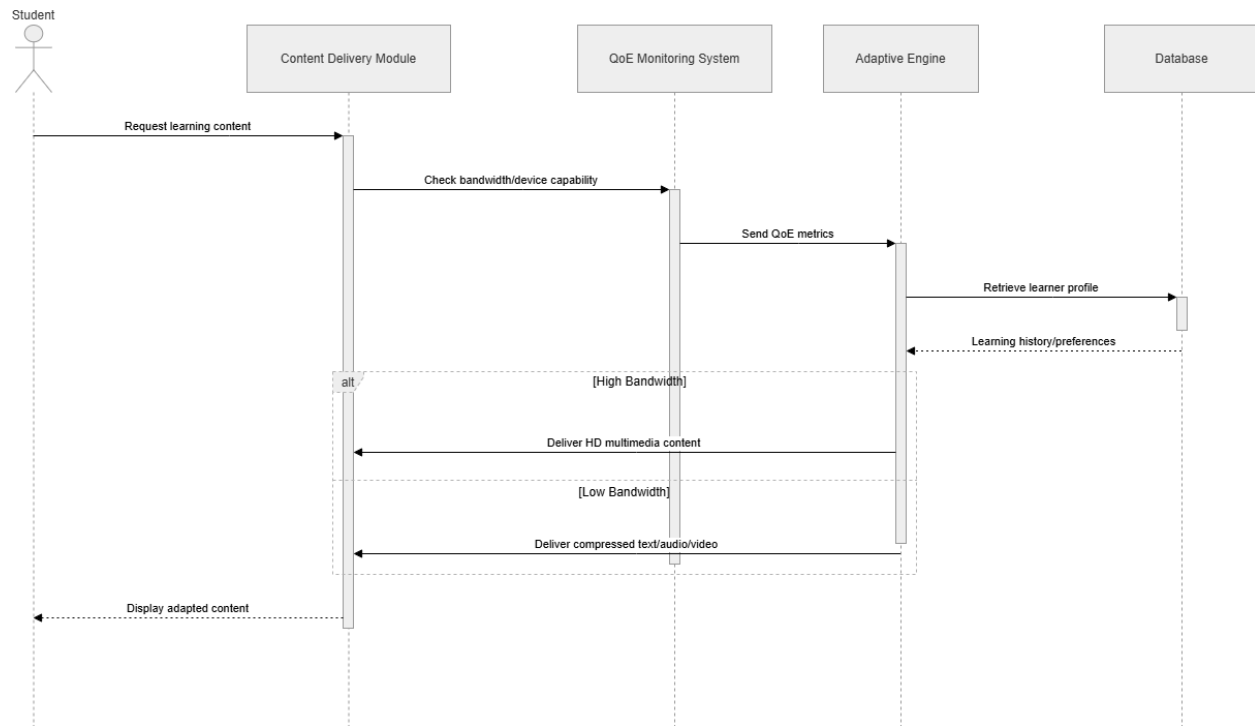


Figure 3: Adaptive Content Delivery sequence diagram

4.3 Offline Content Download Sequence

This diagram illustrates how students obtain lightweight learning resources for offline study. The student initiates a download request, prompting the Content Module to consult the Adaptive Engine for the most appropriate file format. The engine retrieves compressed and optimised content from the database and returns it to the Content Module, which delivers the material to the student's device for offline access.

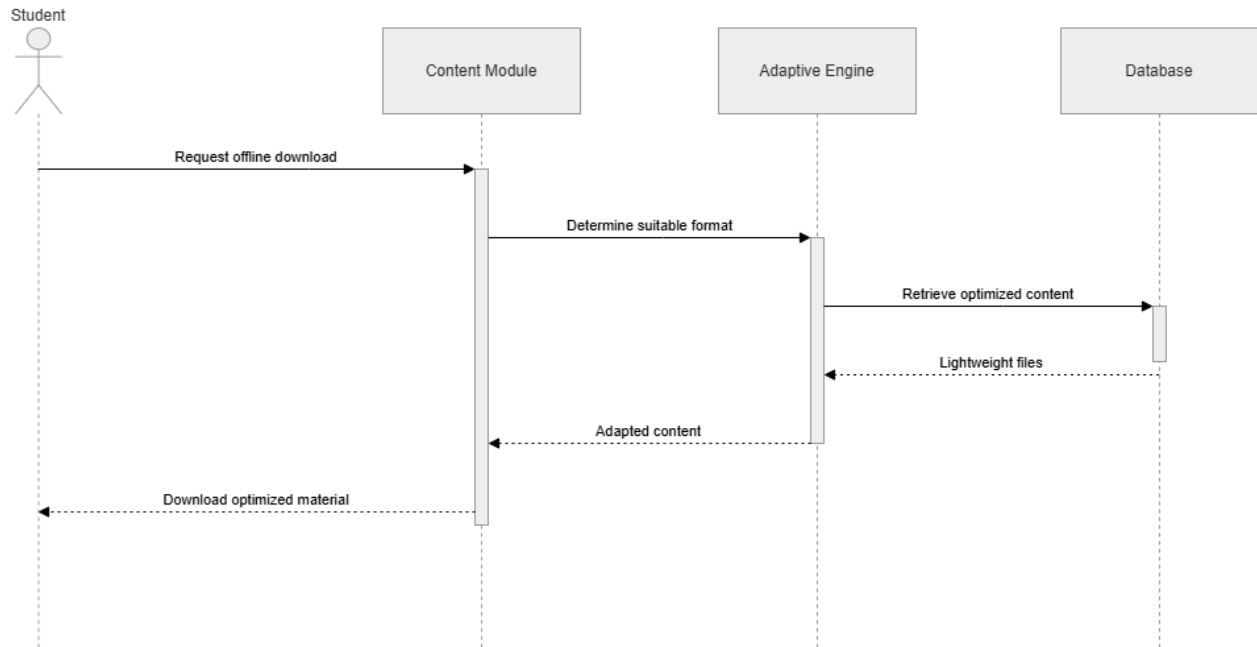


Figure 4: Offline content download sequence diagram

4.4 Adaptive Quiz Recommendation Sequence

This diagram explains the process by which learner performance data is analysed to generate personalised academic recommendations. Following the completion of an assessment, the Assessment Module submits performance data to the Adaptive Engine. The engine analyses the results alongside the student's learning history retrieved from the database and generates tailored recommendations and learning paths, which are subsequently presented to the student.

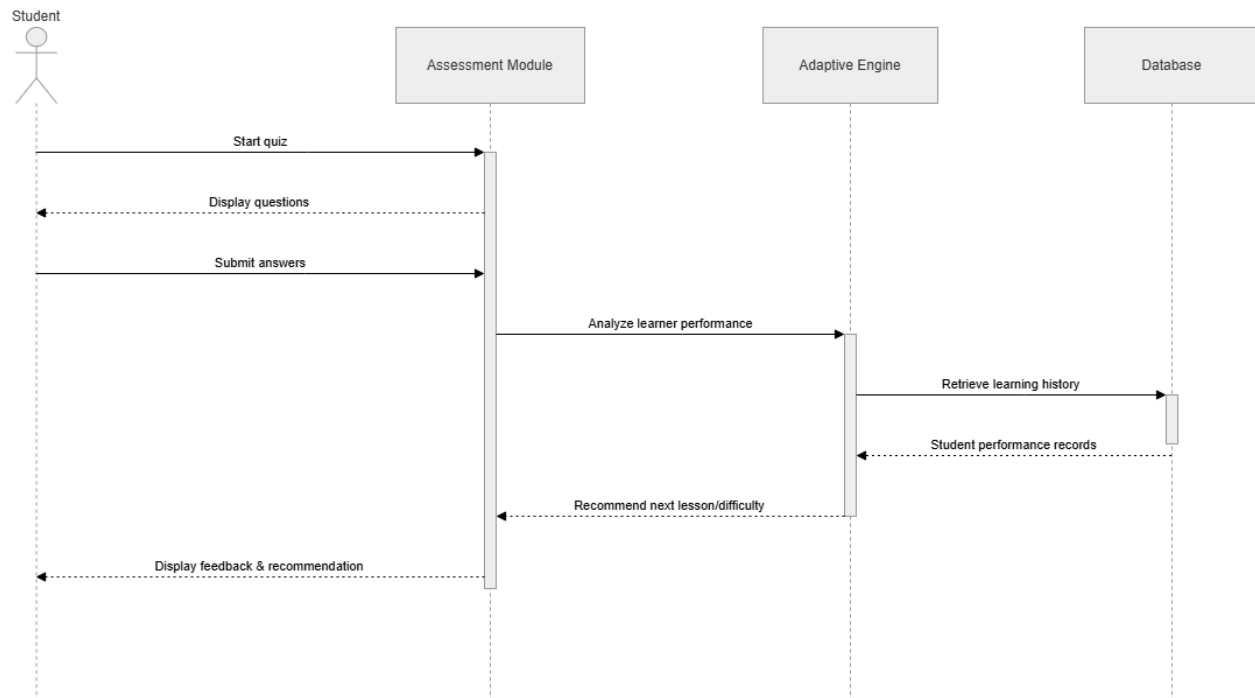


Figure 5: Adaptive quiz recommendation sequence diagram

4.5 QoE Monitoring and Dynamic Adaptation Sequence

This diagram illustrates the continuous monitoring cycle that underpins the platform's adaptive behaviour. The QoE Monitoring System perpetually measures network latency and bandwidth and transmits these metrics to the Adaptive Engine. The engine adjusts content quality in real time based on the received data, ensuring that optimised learning content is delivered to the student without interruption.

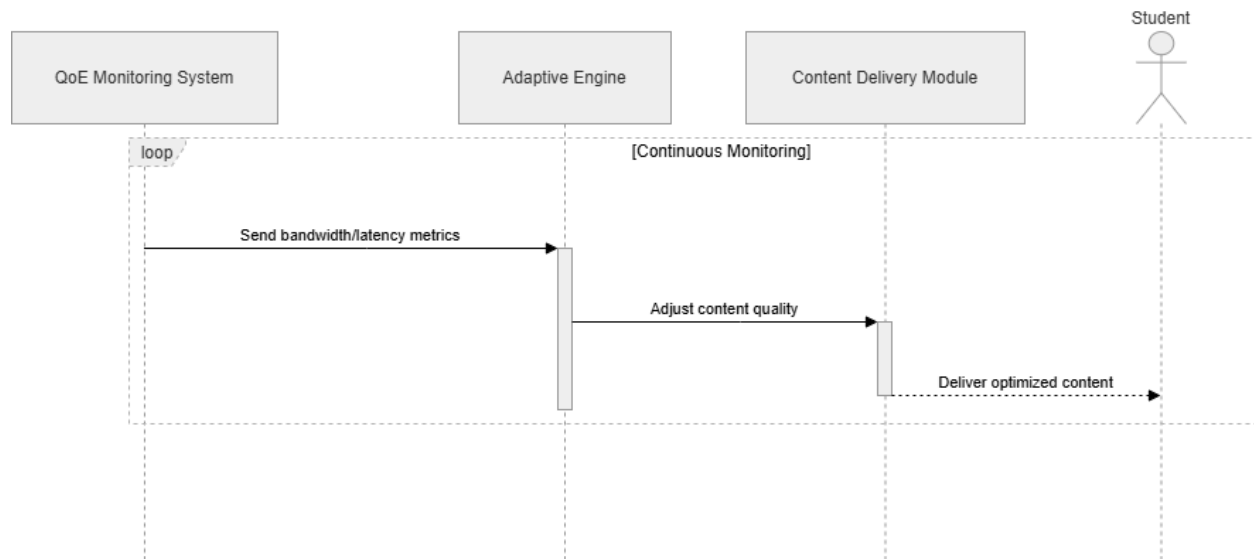


Figure 6: *QoE Monitoring and dynamic adaptation sequence diagram*

4.6 Instructor Content Upload Sequence

This diagram depicts the process through which instructors publish learning materials onto the platform. The instructor uploads content through the system interface, and the Content Management Module stores the material in the database. The Adaptive Engine subsequently generates multiple optimised versions of the content in various formats and quality levels, which are stored in the database. A confirmation notification is returned to the instructor upon successful upload.

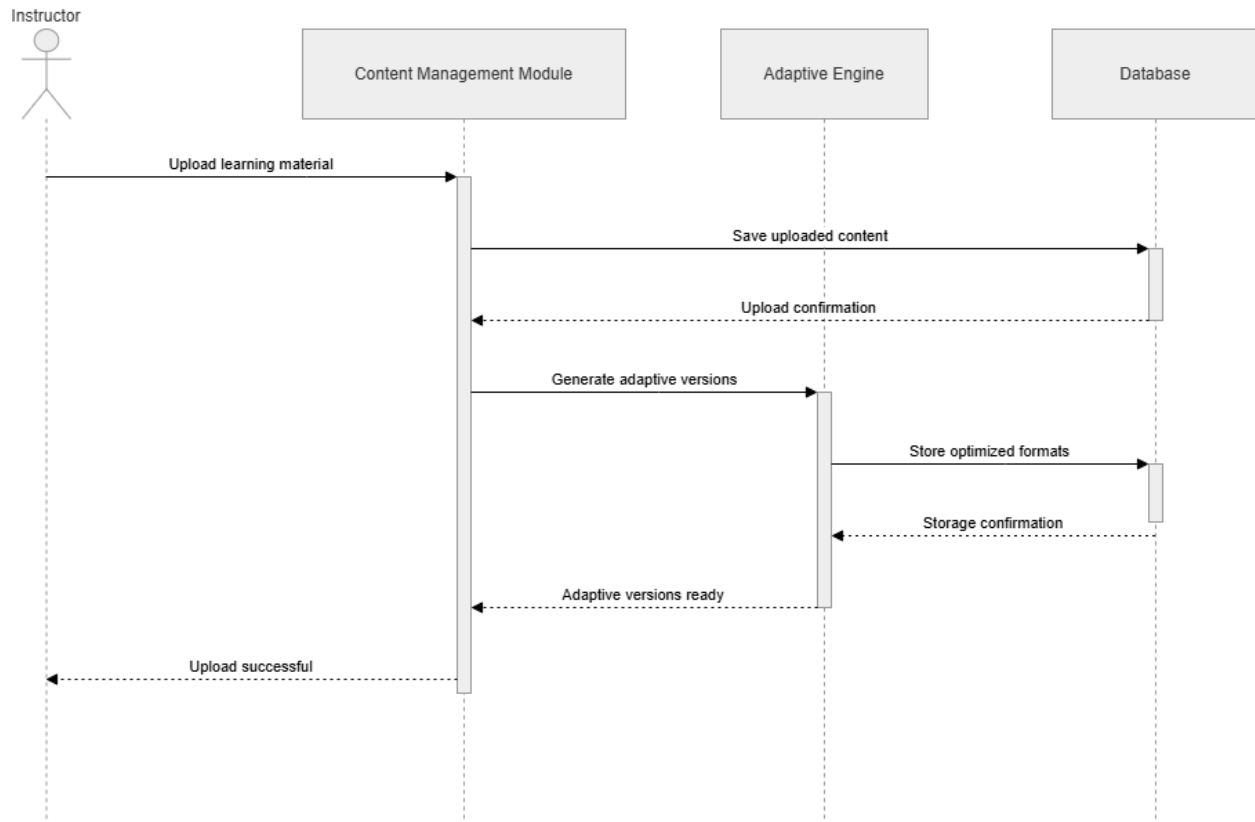


Figure 7: Instructor Content upload sequence diagram

4.7 Administrator QoE Configuration Sequence

This diagram illustrates how administrators manage the adaptation settings and QoE thresholds that govern the platform's content delivery behaviour. The administrator inputs new configuration settings through the admin panel, which communicates the updated parameters to the Adaptive Engine. The engine stores the new settings in the database and returns a confirmation message to the administrator upon successful configuration.

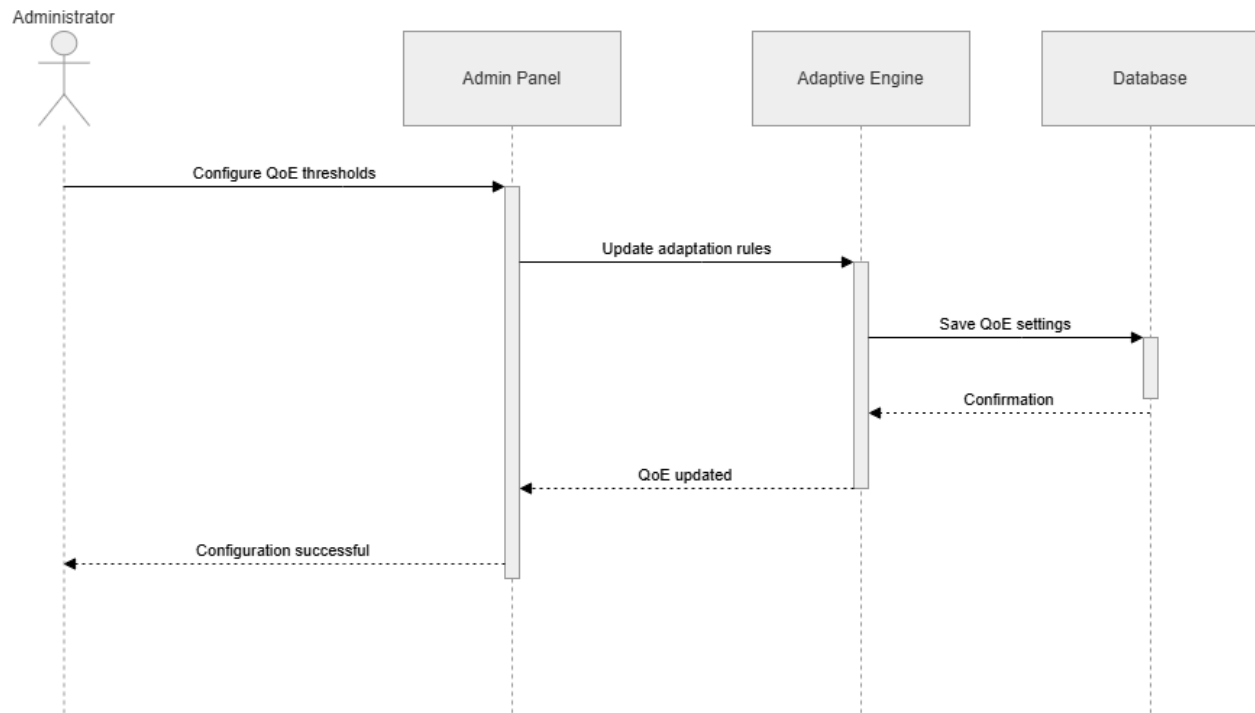


Figure 8:Administrator QoE Configuration sequence diagram

4.8 Student Progress Tracking Sequence

This diagram demonstrates how learner analytics and progress reports are generated and presented. The student requests a progress report through the system interface. The Progress Tracking Module retrieves the relevant learner analytics from the database, processes and analyses the performance data, and renders the student progress dashboard for review.

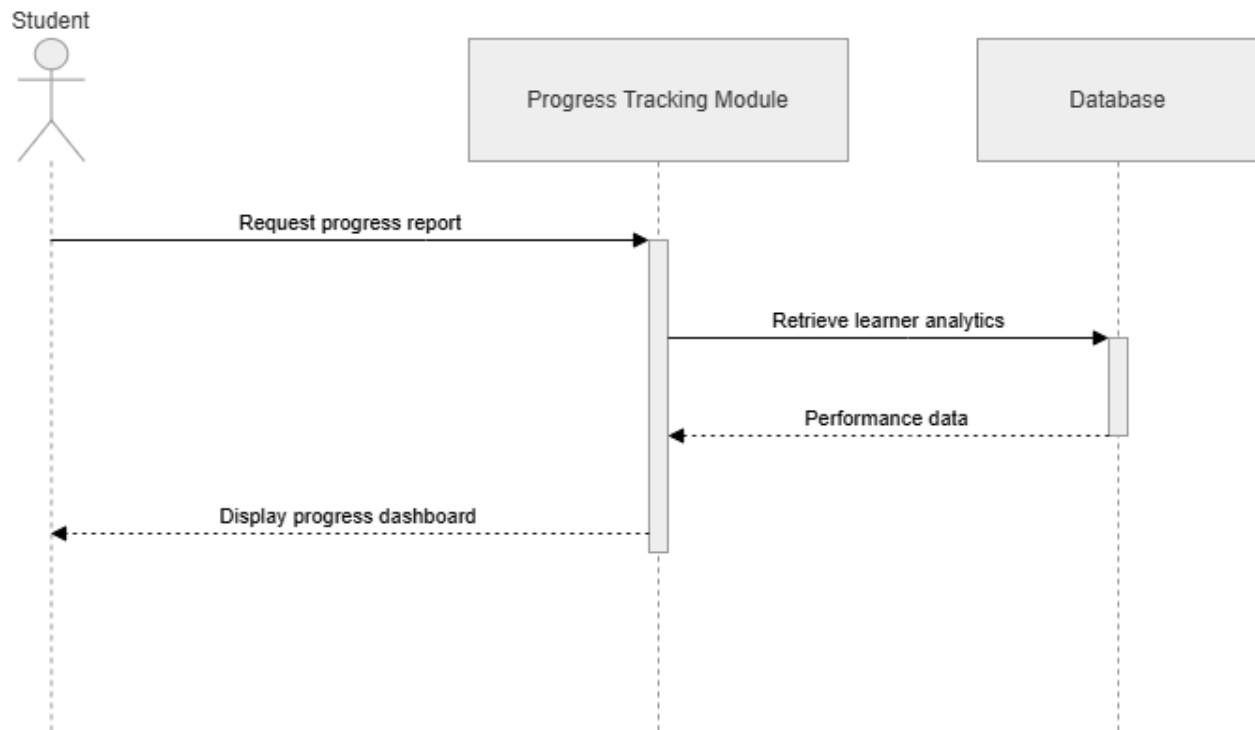


Figure 9: Student progress tracking sequence diagram

5. Class Diagram

5.1 Introduction

A class diagram is a structural UML diagram that represents the static architecture of a system by defining its classes, their attributes and methods, and the relationships between them. For the Adaptive E-Learning Platform, the class diagram reveals how eleven distinct classes collaborate to deliver an intelligent and adaptive learning experience, particularly for students in low-resource environments with limited internet access and basic computing devices.

5.2 Classes and Their Descriptions

5.2.1 User Class

The User class serves as the base class for all user types in the system. Common attributes such as name, email, and password are defined here once and inherited by the Student, Instructor, and Administrator classes, eliminating redundancy. The class handles fundamental operations including user authentication, profile management, and account deletion.

5.2.2 Student Class

The Student class extends the User class and represents the learners on the platform. It stores attributes specific to low-resource contexts, including the student's device type, bandwidth level, and offline content list. These attributes ensure that the system is always informed of each student's technical environment, enabling appropriate content delivery decisions.

5.2.3 Instructor Class

The Instructor class extends the User class and represents educators on the platform. It captures professional information such as qualifications, areas of expertise, and instructor rating. Instructors are able to create and publish courses, upload content in multiple formats, and monitor the academic progress of their students.

5.2.4 Administrator Class

The Administrator class extends the User class and represents the system managers responsible for platform governance. Administrators possess the highest level of access and are responsible for managing user accounts, configuring adaptive system behaviour, and generating analytical performance reports.

5.2.5 Course Class

The Course class is the central entity of the platform. Every piece of learning content and every assessment is associated with a course. It stores critical metadata including the course title, difficulty level, language, and publication status. The Course class serves as the primary connection point between instructors who create courses and students who enroll in them.

5.2.6 Content Class

The Content class manages all learning materials on the platform, encompassing videos, audio recordings, and text documents. Of particular significance is the `adaptFormat()` method, which automatically adjusts the quality and format of content in response to a student's network conditions, ensuring reliable delivery even over slow or unstable connections.

5.2.7 Assessment Class

The Assessment class is responsible for managing all quizzes and formal evaluations on the platform. It stores assessment questions, maximum scores, time limits, and permitted attempt counts. It is also capable of automatically calculating student scores and enabling instructors to review results, making it a critical tool for measuring academic progress.

5.2.8 QoEMonitor Class

The QoEMonitor class is one of the most distinctive classes in this system. It continuously measures the quality of each student's network connection by tracking bandwidth, latency, packet loss, and signal strength, while also identifying the device type and screen resolution in use. All collected data is transmitted to the Adaptive Engine to inform content delivery decisions.

5.2.9 AdaptiveEngine Class

The AdaptiveEngine class constitutes the intelligence of the platform. Upon receiving QoE metrics from the QoEMonitor, it determines the optimal content format and quality level for the student at that moment. It is capable of selecting between high-definition video, standard-definition video, audio-only content, or text-based materials depending on available bandwidth. It also generates personalised learning path recommendations to guide each student's academic progression.

5.2.10 ProgressTracker Class

The ProgressTracker class maintains a comprehensive record of each student's learning journey. It tracks course completion rates, time spent on study, assessments passed, and overall academic scores. This data is of value to both students seeking to monitor their own progress and instructors seeking to evaluate the effectiveness of their courses.

5.2.11 Enrollment Class

The Enrollment class manages the formal process of students registering for courses. It records the enrollment date, current enrollment status, and whether a certificate of completion has been issued. It functions as the official record linking a student to a course throughout the entirety of their learning journey.

5.3 Relationships

5.3.1 Inheritance

The Student, Instructor, and Administrator classes all inherit from the User class, meaning each is a specialised type of user that shares the common attributes and behaviours defined in the parent class. This design promotes code reusability and ensures consistency in how user-related operations are handled across the system.

5.3.2 Composition

The Course class maintains a composition relationship with both the Content class and the Assessment class. Content and assessments are entirely dependent on their parent course for existence. Should a course be deleted from the platform, all associated content and assessments are automatically removed as well, reflecting the strong ownership inherent in this relationship.

5.3.3 Aggregation

The Student class maintains an aggregation relationship with both the Enrollment and ProgressTracker classes. Unlike composition, these classes are capable of existing independently even if the associated student account is removed. Enrollment records and progress data are preserved for historical analysis and reporting purposes. Similarly, enrollment records persist even when a course is no longer actively available on the platform.

5.3.4 Association

The Instructor class is associated with the Course class, as instructors are responsible for creating and managing courses. The QoEMonitor class is associated with the AdaptiveEngine class, as the monitor continuously supplies the engine with real-time network performance data. The AdaptiveEngine is associated with the Content class because it governs how content is formatted and delivered. Additionally, the AdaptiveEngine is associated with the ProgressTracker class as it updates student progress records based on the adaptive content that has been delivered.

5.4 Multiplicity

The multiplicity of each relationship defines the number of instances that may participate in an association. One student may be enrolled in many courses, and one course may have many students enrolled in it. One instructor may create and manage many courses on the platform. One course

may contain many items of content and many assessments. Each student is associated with exactly one progress tracker dedicated to their personal learning journey. Many QoE monitors operating across different student devices collectively feed data into a single centralised adaptive engine for processing.

5.5 Conclusion

The class diagram for the Adaptive E-Learning Platform presents a well-structured and purposefully designed system architecture. Each class fulfils a clearly defined role, and the relationships between classes accurately reflect the operational dynamics of the platform. The QoEMonitor and AdaptiveEngine classes in particular distinguish this system from conventional e-learning platforms by endowing it with the intelligence necessary to respond to each student's unique technical environment. Together, the eleven classes provide a robust and scalable foundation for the implementation of an inclusive, adaptive, and accessible educational platform.

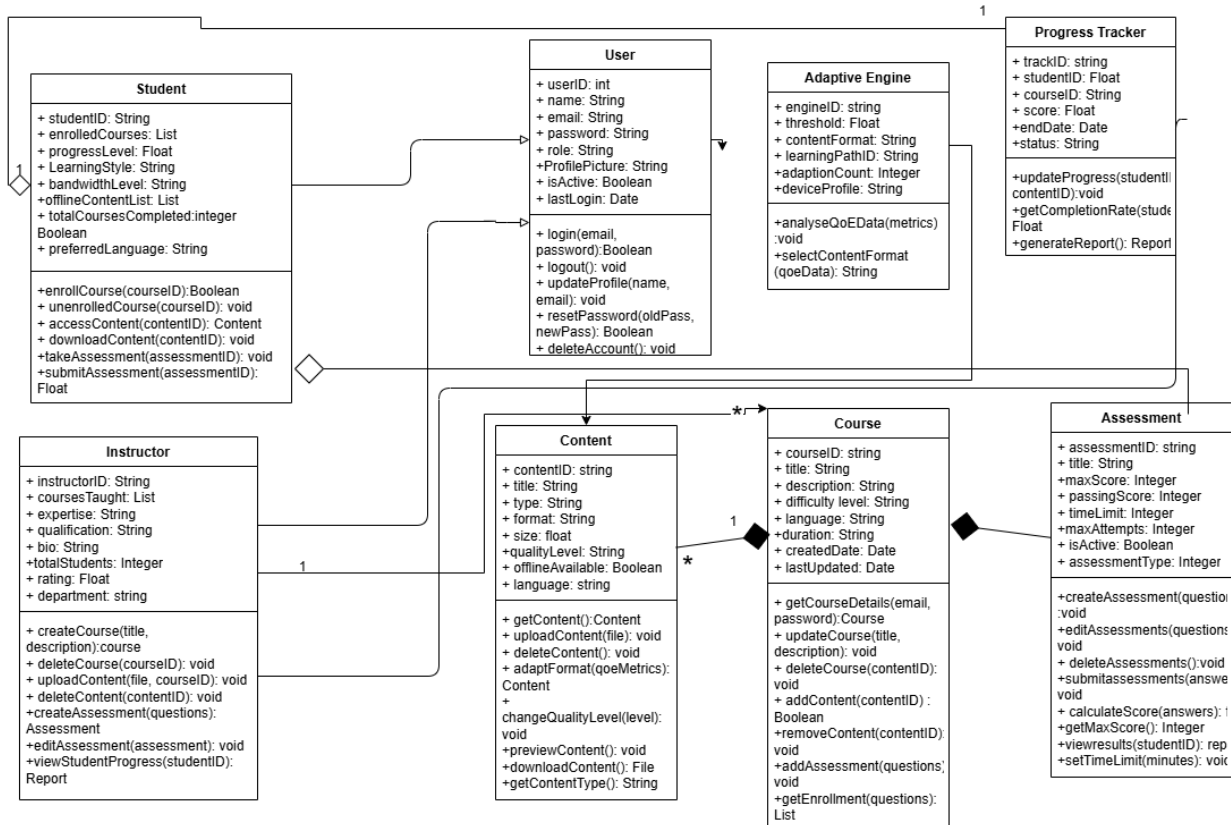


Figure 10: System Class diagram

6. Deployment Diagram

6.1 Introduction

A deployment diagram is a structural UML diagram that illustrates how software artefacts, including executables, databases, and configuration files, are distributed across the physical or virtual nodes that constitute a system's infrastructure. It addresses the fundamental questions of what runs where and how the various components communicate at runtime. For the Adaptive E-Learning Platform, the deployment diagram is of particular importance as it demonstrates how the system's architecture has been designed to accommodate the connectivity constraints faced by students in low-resource settings.

6.2 System Overview and Component Mapping

The platform's components are distributed across six deployment nodes. The Student Device hosts the mobile application, the QoE Monitor module, and the offline content cache. The Instructor Device hosts the mobile application and the content upload module. The Admin Browser hosts the administrative web dashboard. The Cloud Application Server hosts the REST API, Adaptive Engine, Authentication Service, QoE Service, Progress Tracker, and Push Notification Service. The Database Server hosts the Application Database and the QoE Logs Database. The CDN and Cloud Storage node hosts video, audio, and PDF learning materials for optimised delivery.

6.3 Deployment Architecture

6.3.1 Three-Tier Structure

The deployment architecture follows a three-tier pattern. The Client Tier comprises the mobile devices used by students and instructors, as well as the desktop browser used by the administrator. User-facing applications execute at this tier. The Application Server Tier consists of a cloud-hosted node running all backend service artefacts and contains all business logic. The Data and Content Tier comprises a separate database server and CDN responsible for persisted data and media delivery, neither of which is directly accessible from the public internet. This separation of tiers enables independent scaling, enhances security, and simplifies ongoing system maintenance.

6.3.2 Protocol Choices

All client-to-server communication, including login, course access, quiz submission, and content retrieval, is conducted over HTTPS using TLS 1.2 or 1.3, which is universally supported on mobile platforms. Continuous QoE telemetry from the mobile client to the QoE Service is transmitted via WebSocket, which was selected over HTTP polling to eliminate repeated connection overhead, a critical consideration in environments where mobile data is costly. Communication between the application server and the database server occurs over SQL and TCP within the cloud provider's internal network and is not exposed to the public internet. Media content is fetched by the mobile client directly from the nearest CDN edge node using a signed URL provided by the API, bypassing the application server for large file transfers. Reading and writing to the on-device offline cache involves only local input and output operations with no network protocol.

6.4 Design Decisions for the Low-Resource Context

Several deployment decisions were made specifically in response to the constraints identified during the requirement gathering phase. To address unstable and slow connectivity, the QoE Monitor transmits live telemetry via WebSocket, enabling the Adaptive Engine to switch content to a lower-bandwidth format automatically and in real time. To mitigate high mobile data costs, content is served directly from the CDN via signed URLs, avoiding the routing of large files through the application server, and adaptive quality reduction further reduces data consumption. To accommodate intermittent connectivity, an offline cache artefact on the student device allows pre-downloaded content to be accessed without any network connection, with progress synchronised upon reconnection. To support low-end Android devices, all computationally intensive processes, including adaptive decision-making and analytics, are executed on the server, keeping the mobile application as a thin client. Content may also be delivered in compressed audio or PDF format to accommodate limited device capabilities. Finally, to ensure accuracy across varied network types including 2G, 3G, and 4G connections, the QoE Monitor measures actual throughput and latency directly rather than relying on the operating system's reported network type.

6.5 Conclusion

The deployment diagram for the Adaptive E-Learning Platform demonstrates a thoughtfully engineered infrastructure designed with the realities of low-resource settings at its core. The three-

tier architecture, combined with adaptive quality controls, offline caching capabilities, and CDN-based media delivery, ensures that the platform remains performant, secure, and accessible to all students regardless of the limitations of their network connectivity or device hardware.

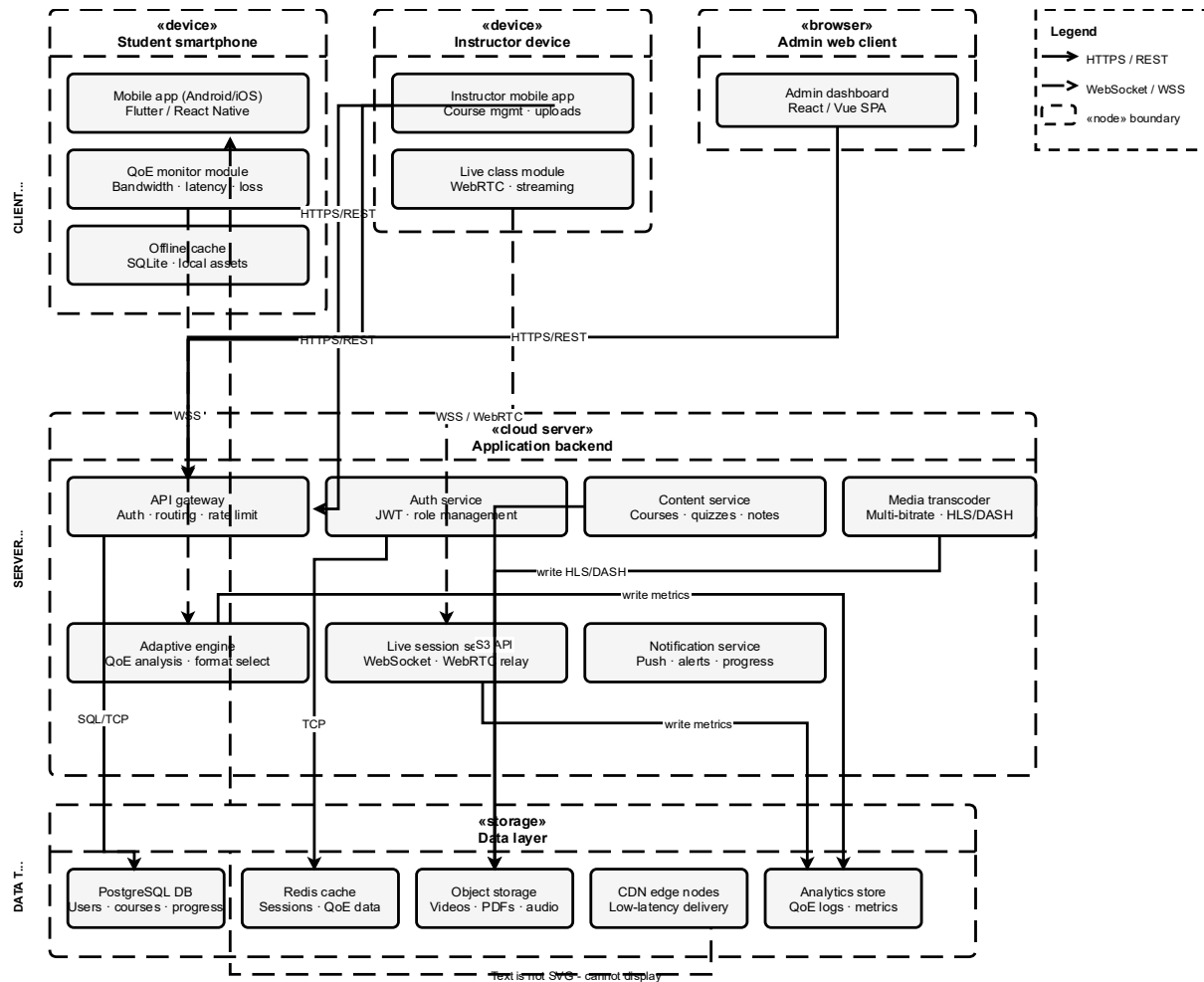


Figure 11: System Deployment diagram